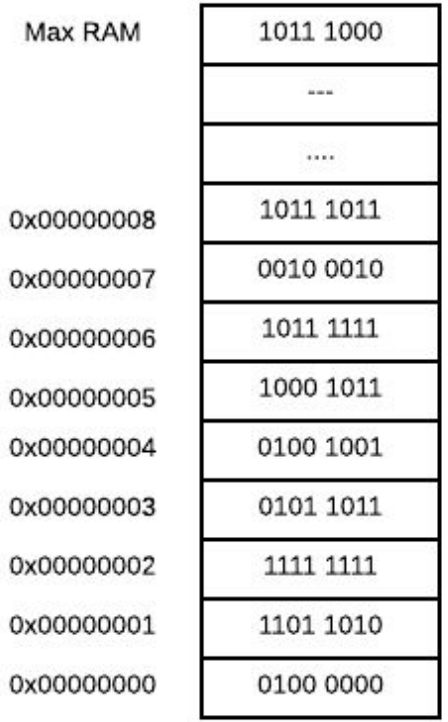
Operating Systems and Parallel Computing

Lecture 6

Prof. Matteo Turilli

Memory Abstraction

- Modern computers have multiple types of physical memory with different speed and capacity
- Here we focus on the random access memory (RAM), the volatile memory the CPU uses to execute programs
- Before 1980 personal computers had no memory abstraction
- Every program could directly access the physical memory
- Programmers memory model was:
  a. A set of addresses from 0 to some maximum
  b. Each address referred to a cell containing N bits, commonly eight
  c. An instruction to move bits to an address in memory
- What problems do you see with this model?



Direct Access To RAM

- It is not possible to have two running programs in memory at the same time with a direct memory access model.
- Both programs can write the same locations, erasing whatever value the other program was previously storing there.
- Nothing would work and both programs would crash almost immediately.
- Only one process at a time can be running:
  a. A user types a command requesting the execution of a program
  b. The OS copies the requested program from nonvolatile storage (e.g., HD) to memory (RAM) and executes it, i.e., creates a process that uses the ram directly with no memory abstraction
  c. When the process finishes, the operating system displays a prompt character and waits for a user new command.
  d. When the operating system receives the command, it loads a new program into memory, overwriting the first one.

Direct Access To RAM

- It is not possible to have two running programs in memory at the same time with a direct memory access model.
- Both programs can write the same locations, erasing whatever value the other program was previously storing there.
- Nothing would work and both programs would crash almost immediately.
- Only one process at a time can be running:
  a. A user types a command requesting the execution of a program
  b. The OS copies the requested program from nonvolatile storage (e.g., HD) to memory (RAM) and executes it, i.e., creates a process that uses the ram directly with no memory abstraction
  c. When the process finishes, the operating system displays a prompt character and waits for a user new command.
  d. When the operating system receives the command, it loads a new program into memory, overwriting the first one.
- Is parallelism possible at all with this memory access model?

Direct Access To RAM

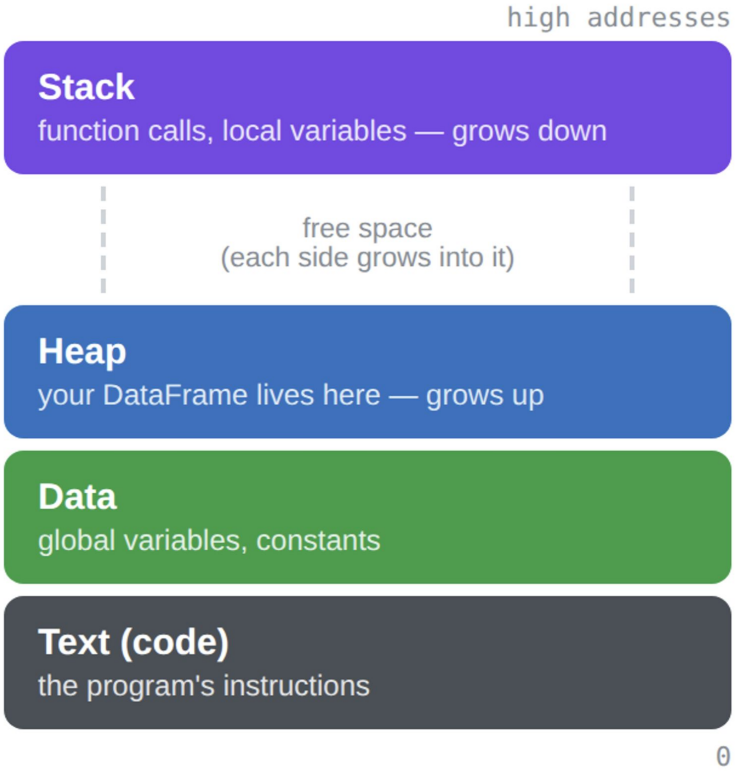
- It is not possible to have two running programs in memory at the same time with a direct memory access model.
- Both programs can write the same locations, erasing whatever value the other program was previously storing there.
- Nothing would work and both programs would crash almost immediately.
- Only one process at a time can be running:
  a. A user types a command requesting the execution of a program
  b. The OS copies the requested program from nonvolatile storage (e.g., HD) to memory (RAM) and executes it, i.e., creates a process that uses the ram directly with no memory abstraction
  c. When the process finishes, the operating system displays a prompt character and waits for a user new command.
  d. When the operating system receives the command, it loads a new program into memory, overwriting the first one.
- Is parallelism possible with this memory access model? Yes, with modern threads. Why?

Direct Access To RAM

- It is not possible to have two running programs in memory at the same time with a direct memory access model.
- Both programs can write the same locations, erasing whatever value the other program was previously storing there.
- Nothing would work and both programs would crash almost immediately.
- Only one process at a time can be running:
  a. A user types a command requesting the execution of a program
  b. The OS copies the requested program from nonvolatile storage (e.g., HD) to memory (RAM) and executes it, i.e., creates a process that uses the ram directly with no memory abstraction
  c. When the process finishes, the operating system displays a prompt character and waits for a user new command.
  d. When the operating system receives the command, it loads a new program into memory, overwriting the first one.
- Is parallelism possible with this memory access model? Yes, with modern threads. Why? Because threads share the memory of their parent process.

Address Space: Everyone Gets “Fake” Memory

- Each process gets a private **address space** from 0 to up or beyond the RAM size
- The program only uses these private addresses and it never sees a physical one.
- Address 5000 in our notebook and our browser are different physical bytes.
- Each process gets its own private address space organized in segments/regions:
  a. Heap: holds the program data, e.g., a Panda DataFrame, and grows the more data are loaded
  b. Stack: holds program’s working data (e.g., variables) and grows at runtime
- Useful illusion: how does the OS do it?!



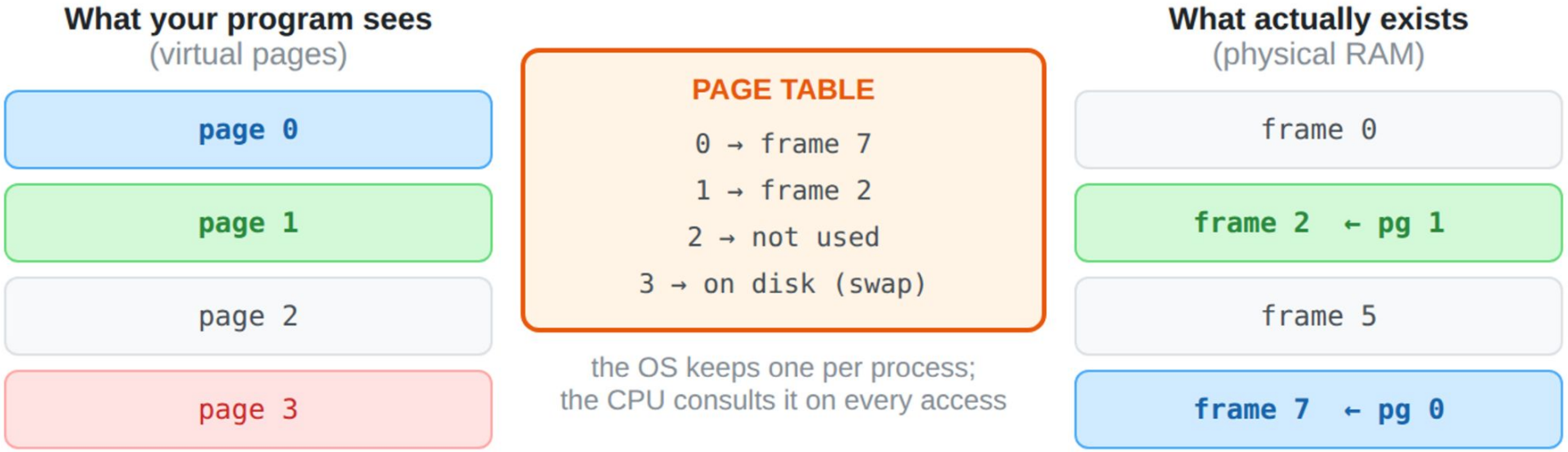
Pages: Memory in Fixed-Size Blocks

- The OS hands out an address space to each process made of **pages**, typically each page is 4 KB.
- The OS maintains exclusive access to the physical RAM and organizes the physical RAM in a list of frames of the same size as the pages.
- The OS keeps a page table for each process to track which virtual page maps to which physical frame.
- Note: A page does not have to be in RAM!
  a. It may be unused by the process, e.g., input data that still need processing
  b. Parked on disk, e.g., when unused by the process or when the OS is context switching processes.
- The CPU consults this mapping on every memory access, billions of times/s.

Swap: Extending RAM On Disk

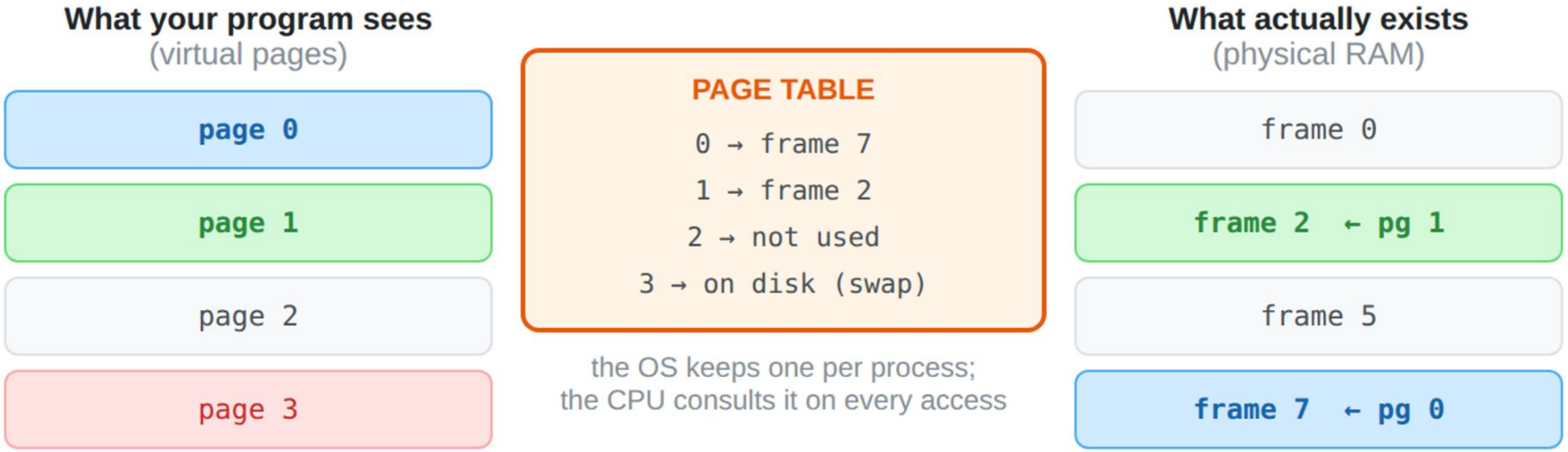
- Swap extends system memory by using NVMe SSD/HDD as a spillover zone.
- Memory management technique where the OS temporarily moves active/inactive data from physical RAM onto storage (known as swap space).
  a. Active data: we run out of RAM and instead of killing the processes, we use the much slower storage
  b. Inactive: Relocating inactive background memory pages from physical RAM to storage drives frees up premium RAM for active tasks at the cost of slower storage speeds.
  c. Virtualized memory: using the swap to run processes that require more memory than the RAM
- We want as much RAM as possible in our system:
  a. The more RAM we have the less the OS has to use the swap (if at all)
  b. The OS can cache input data directly in RAM while moving stagnant pages to the swap.
  c. The OS can create a so-called zram space into the RAM to serve the process swapping logic instead of using a much slower swap. Note that zram trades CPU cycles for memory space.
- Note: relying too heavily on swapping on disk wears out hardware, forcing non-stop read/write cycles that degrade the lifespan of a solid-state drive.

Pages: Memory in Fixed-Size Blocks



- Let's wear our performance hat: Do you see anything worrying with this approach?

Pages: Memory in Fixed-Size Blocks



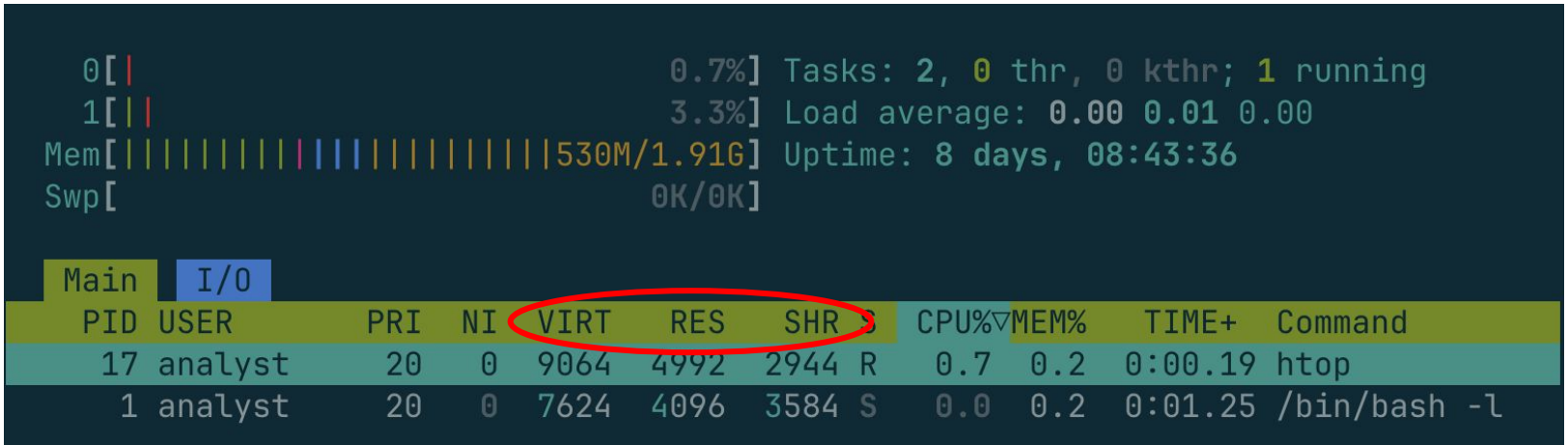
- Let's wear our performance hat: Do you see anything worrying with this approach?
- Every memory access requires a translation operation from pages to frame (1 operation) before actually accessing the data of that frame (1 operation)
- Billions of times a second = serious overhead!

Why Is Translation Not Super Slow?

- The CPU keeps a small, very fast cache of recent translations called the TLB (Translation Lookaside Buffer).
- Programs touch the same few memory pages over and over, so the TLB hit rate is typically well above 99%.
- This is the same principle as caching everywhere else:
  a. Exploit locality: the TLB cache is local and physically near to the core/CPU
  b. Optimize the common case: processes hits over and over the same page.
- When code jumps around memory randomly, the TLB cannot help. This is one reason scattered accesses are much slower than sequential ones. More in Part II.

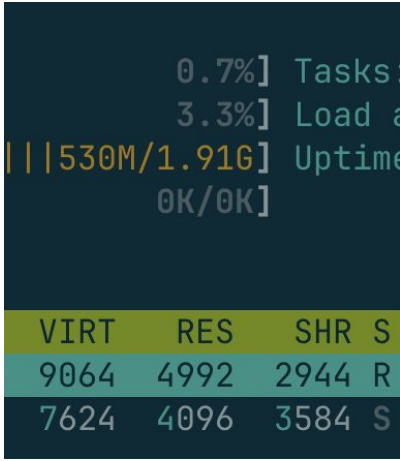
Back To `htop`

- VIRT (virtual): Everything the process thinks it has access to (i.e., address space).
- RES (resident): The actual physical RAM currently occupied by the process. This is the number that matters most for system performance.
- SHR (shared): The portion of RES that is shareable with other processes (e.g., public system libraries).
- **Question**: Wait what?! Processes cannot share their memory!



Back To `htop`

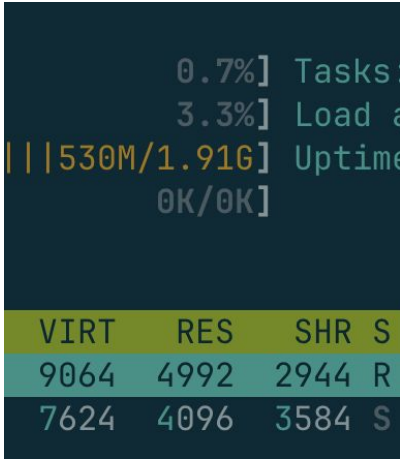
- VIRT (virtual): Everything the process thinks it has access to (i.e., address space).
- RES (resident): The actual physical RAM currently occupied by the process. This is the number that matters most for system performance.
- SHR (shared): The portion of RES that is shareable with other processes (e.g., public system libraries).
- **Question**: Wait what?! Processes cannot share their memory!



- Answer: processes cannot but the OS can :) It will map some of the address space on the same frame, without the process knowing it
- **Question**: how can a process be promised more memory than the machine has?

Back To `htop`

- VIRT (virtual): Everything the process thinks it has access to (i.e., address space).
- RES (resident): The actual physical RAM currently occupied by the process. This is the number that matters most for system performance.
- SHR (shared): The portion of RES that is shareable with other processes (e.g., public system libraries).
- **Question**: Wait what?! Processes cannot share their memory!



- Answer: processes cannot but the OS can :) It will map some of the address space on the same frame, without the process knowing it
- **Question**: how can a process be promised more memory than the machine has?
- Answer: the OS is an optimistic gambler. It assumes that when a program asks for memory, it is asking for a worst-case scenario and won't actually use all of it.

What If The OS Loses The VIRT Gamble?

- If the process starts writing data to every page it was promised, the transition from VIRT to RES forces the OS into a resource crunch.
- Performance Drop (thrashing phase):
  a. As the process fills up the physical RAM, the OS scrambles to free up space using our storage drive.
  b. The OS spends its time moving pages back and forth, between RAM and disk.
  c. Result: Our system enters a state called thrashing. The mouse cursor freezes, UI elements stop responding, and the entire computer slows to a near-total crawl.
- Out-Of-Memory (OOM) Killer strikes:
  a. If the swap space fills up too, the next time our process tries to write to a new memory address, the CPU triggers a Page Fault but the OS has zero physical frames left to hand out.
  b. To save itself from a total system crash, the OS activates the OOM killer.
  c. The OS calculates a "badness score" for every running application and our process that is actively chewing through massive amounts of memory will be immediately terminated.

This is an interactive class

- Start JupyterLab: `docker compose up jupyter`
- Copy the `http://localhost:8888/lab?token=...` link into your browser.
- Download `lecture_06_notebook.ipynb` from our Blackboard and move it into the work directory of our Docker container

The Pattern, Again

- Session 1: the loop took more time to execute because it did many small operations instead of few large ones.
- Session 5: reading one byte at a time cost much more time for the same reason, one layer down compared to Session 1.
- Session 6: oversized dtypes cost memory because every row pays for precision it does not need.
- Lesson learned: Performance comes from knowing:
  - What resources our application is asking for
  - How the operating system is managing the computing resources requested.

Exercises and Homework

- In the container, in `06_memory.ipynb`:
  - Run the notebook. What is your shallow-vs-deep ratio, and your dtype saving?
  - Add a column of your own (e.g. df["note"] = "ok") and measure it deep. Why is such a short string so expensive?
- In a terminal:
  - Run htop beside the notebook. Watch VIRT and RES as you run the `np.zeros` cell.
  - `cat /proc/meminfo | head -3` : how much memory does the container think it has? Compare with our 2 GB limit.
- Remember: VIRT is a promise, RES is reality, and dtypes are free performance.
- Next time: what happens when the promises come due and there are no free pages left? Swapping, thrashing, and the process that gets killed.